*Méndez Reyes Juan Diego

Parcial 1 • Correcciones

Pregunta 1 - Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un gemplo pequeño de código ¿Qué es la abstracción al modelar un objeto? Da un dato que conservaría y otro que omitirías según el contexto.

La abstracción es quedarse con lo esencial para el problema; un ejemplo para un perro en una veterinaria: solo se guarda su raza y vacunas se omite el color de su plato.

Consiste en guardar solo los datos de un objeto que sirven para nuestro programa y descartar el resto.

Pregunta 2 - ¿Para que sirve el constructo y en que se distingue de un metodo común en su declaración? El constructor sirve para inicializar un objeto y asignar los valores iniciales a sus atributos al momento de crearlo.

Se distingue de un método común en su declaración porque se llama exactamente igual que la clase y no tiene tipo de retorno (ni siquiera void)

Pregunta 3 ¿Que impide final en una variable? Si guarda una referencia a un arreglo ¿impide cambiar sus elementos? Final impide volver a asignar la variable después de inicializarla, si guarda un arreglo, no se puede cambiar la referencia, pero sí sus elementos.

Pregunta 4 ¿Porqué == no basta para comparar el contenido de dos string? Explica que usaría para comparar sus textos. El operador == compara las referencias en memoria de los objetos (si apuntan al mismo objeto en memoria o su identidad), no sus caracteres.

Para comparar el contenido o texto de dos cadenas se debe utilizar el metodo .equals()

Pregunta 5 ¿En qué se distinguen while y do-while? Explica que ocurre si la condición es falsa desde el inicio? ¿While revisa la condición antes de ejecutar el bloque de código. Si es falsa desde el inicio, da 0 vueltas (no se ejecuta nada), • Do-while revisa la condición despues de ejecutar el bloque. Si es falsa desde el inicio, se ejecuta al menos una vez antes de salir.

Pr 1 ¿Que imprime?

```
Int [][] tabla = {{a,4,7}, {2,6,8}};
int [] fila = tabla [0];
int total = 0;
for (int i=0; i < fila.length; i++) {
fila [i] = fila [i] / 2;
 total += tabla [0][i];
System.out.println (i + ":" + total);
}
System.out.println ((double) (total/2));
System.out.println("p" + fila[0] + tabla[1] [0]);
```

Vuelta	i	fila [i] antes	fila[i]=fila[i]/2	tabla[0][i]
1	0	9	9/2 = 4	4
2	1	4	4/2 = 2	2
3	2	7	7/2 = 3	3

Tota += tabla [0][i]
0 + 4 = 4
4 + 2 = 6
6 + 3 = 9

Salida en la consola (i+":"total)
0 : 4
1 : 6
2 : 9

Escribe una descripción en vez de la salida exacta. Reconoce que la cuarta líne es total entre dos y "P" concatenada con fila y tabla en la quinta

R2 (a) ¿Compila, truena o funciona?

```
int[] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
System.out.println(total);
```

* Compila pero truena. Con i<=datos.length el ciclo intenta leer datos[2], fuera del arreglo (ArrayIndexOutOfBoundException) y el println nunca se ejecutan.

En Java los índices de un arreglo van desde 0 hasta length - 1
Como el arreglo dato tiene 2 elementos {3, 6}, los índices validos son 0 y 1 (el datos.length) es 2

Vuelta 1 (i=0) total += datos[0] → total = 0+3 = 3
Vuelta 2 (i=1) total += datos[1] → total = 3+6 = 9
Vuelta 3 (i=2) como la condición usa <= en lugar de <
el ciclo intenta ejecuta datos[2]

Al no existir el índice [2] y Java truena el programa y arroja el error ArrayIndexOutOfBoundsException

R2 (b) Compila o truena o funciona?

```
Integer piezas = 6
System.out.println(piezas+2);
System.out.println("P" + piezas);
```

Funciona
Salida exacta 8/P6

Integer si existe en Java; es la clase envolvente (wrapper class) del tipo primitivo int, gracias al autoboxing, Java asigna automaticamente el valor 6 al objeto piezas.

- Primer línea de salida
  System.out.println(piezas+2);
  Java realiza un unboxing (desempaquetado) automatico convirtiendo piezas de integer a int para realizar 6+2 = (8)
- Segunda línea
  es (P6) se realiza una concatenación de cadenas Java convierte la variable a texto.

R2 ⓒ ¿Compila, truena, funciona?

```
Static int medir (int valor) {
    if (valor > 0) return valor;
}
```

No compila Cuando valor es cero o negativo no hay return y

① En Java, si un método se declara con un tipo de retorno distinto de void (en este caso int medir(...)), la regla del compilador exige que todos los caminos posibles de ejecución garanticen un valor de retorno (missing return statment

Java exige que todos los caminos de un metodo int devuelvan un valor

② Línea que causa el error
Todo el bloque del método static int medir (int valor) {...}

③ Motivo
El return solo se encuentra dentro de la condición if (valor > 0), si el parametro valor llega a ser cero o un número negativo (valor ≤ 0), el bloque if no se ejecuta y el método llega al final de la llave de cierre } sin haber retornado ningun int
El compilador detecta esta posibilidad y rechaza compilar el programa

R3. Modelado: base, reto, y justificación

enum Estado {APAGADO, ENCENDIDO, PARPADEO}

Codigo Lampara.java

```
Public class Lampara {
    // Atributos
    Private final String etiqueta;          // Constante final por lampara
    Private Estado estado;                  // Atributo mutable del estado
    Private static int totalLamparas=0;     // Atributo static para contar
    // Constructor
    Public Lampara (String etiqueta) {
        this.etiqueta = etiqueta;
        this.estado = Estado.APAGADO;       // Nace Apagada
        totalLamparas++;                    // Incrementa el contador global
}
    //Metodo de cambio (encender)
    public void encender () {
        this.estado = Estado.ENCENDIDO;
    }
    // Metodo de consulta
    public Estado getEstado() {
    return this.estado;
    }
}
```

* El atributo de estado es de tipo Estado (del enum) y nace con el valor Estado.APAGADO.

R4 Código con repetición
Devuelve la posición de la menor temperatura usando for si se repite devuelve la primera. Los índices empiezan en 0
El arreglo no es null, tiene al menos un elemento y puede incluir negativos No lo modifiques
Static int posicionMinima(int[] temperaturas)
Ejemplos (argumentos → retorno);
{8,3,5} → 1 · {7} → 0 · {-2,-8,-1} → 1

```
Stati int posicionMinima(int[] temperaturas){
int posicion = 0;
for(int i=1 ; i < temperaturas.length ; i++){
  if ( temperaturas[i] < temperaturas[posicion]{
            posicion = i ;
    }
}
  return posicion;
}
```

Ejemplo 1
* Inicia = posicion = 0 (Apunta a 8)
i=1 temperaturas[1] (3) es menor que temperaturas[0] (8), por lo que posicion cambia a 1
o i=2 temperaturas[2] (5) no es menor que temp[1] (3) posicion se queda en 1

Se usa < estrictamente en lugar de <= para que, si el número menor se repite mas adelante, se conserve la primera posición encontrada.

Se inicia i=1 porque la casilla 0 ya se toma como la posicion inicial del mínimo candidato

R5 Codigo de/con seleccion

```
Static String mantenimiento (int ciclos) {
    if (ciclos <= 0) {
            return "Invalido";
    } else if (ciclos % 24 == 0) {
        return "completo";
    } else if (ciclos %. 12 == 0) {
        return "Revision";
    } else if (ciclos %. 4 == 0) {
        return "Basico";
    } else {
        return "Ninguno";
    }
}
```

Menor o igual que 0 devuelve invalido

Si ciclos es positivo y ciclos %. 24 == (el residuo de la div es 0) retorna completo

Si no, divisible entre 12

Si no, divisible entre 4 devuelve "Basico"

Si el numero es mayor a 0 pero no → divisible exactamente con 24, 12 ni 4 cae en el bloque else final y retorna "Ninguno"

Cadena if/else if/else garantiza que se aplique unicamente la primera regla que corresponda en el orden exacto especifico por la tabla

return = devuelve las cadenas.